CONSOLIDATED REPORT

10 Python Case-Study Projects

Source code • Test cases • Sample output captures • Observations

Important note
The console panels in this report are illustrative output captures recreated from the sample code and data. They are not screenshots taken from your own computer. If your instructor requires genuine IDE/terminal screenshots, run the programs and replace or supplement these panels.

Contents

1. Attendance Recovery Computation

2. ATM Transaction Sequence Simulation

3. Student Performance Trend Analysis

4. Repeated Transaction Detection

5. Password Policy Validation Engine

6. String-Based Examination Answer Evaluation

7. Text Compression Using Character Runs

8. Student Marks Matrix Analysis

9. Food Waste Monitoring System

10. Energy Conservation Before-and-After Analysis

Testing note

The test cases document representative normal, invalid, and boundary inputs. Expected behavior is based on the sample programs shown in this report. Run each program in Python 3 to verify actual behavior before submission.

1. Attendance Recovery Computation

Objective

Calculate current attendance and additional attended classes needed to reach 75%.

Source code

```
total = int(input("Enter total classes conducted: "))
attended = int(input("Enter classes attended: "))
if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = attended / total * 100
    print(f"Current attendance: {percentage:.2f}%")
    if percentage >= 75:
        print("You are eligible.")
    else:
        extra = 0
        while attended / total * 100 < 75:
            total += 1
            attended += 1
            extra += 1
        print("Additional classes required:", extra)
        print(f"Final attendance: {attended / total * 100:.2f}%")
```

Test cases

Input / condition	Expected result
100 classes, 60 attended	60%; needs 60 more attended classes to reach 75%.
80 classes, 60 attended	75%; eligible.
0 total classes	Invalid input.

Sample output screenshot (illustrative)

```
Enter total classes conducted: 100
Enter classes attended: 60
Current attendance: 60.00%
Additional classes required: 60
Final attendance: 75.00%
```

Observation / limitation

Assumes all future classes are attended.

2. ATM Transaction Sequence Simulation

Objective

Simulate deposits and withdrawals, reject invalid amounts, and summarize transactions.

Source code

```
balance = 5000
transactions = []
print("Starting balance:", balance)
print("Enter D for deposit, W for withdrawal, X to stop")
while True:
    choice = input("Enter transaction type: ").upper()
    if choice == "X":
        break
    if choice != "D" and choice != "W":
        print("Invalid transaction type")
        continue
    amount = float(input("Enter amount: "))
    if amount <= 0:
        print("Amount must be positive")
    elif choice == "D":
        balance += amount
        transactions.append(("Deposit", amount))
        print("Deposit successful")
    elif amount > balance:
        print("Insufficient balance")
    else:
        balance -= amount
        transactions.append(("Withdrawal", amount))
        print("Withdrawal successful")
print("\nTransaction Summary")
for kind, amount in transactions:
    print(kind, ":", amount)
print("Final balance:", balance)
```

Test cases

Input / condition	Expected result
Deposit 1000; withdraw 500	Final balance 5500.0.
Withdraw 6000	Insufficient balance; balance stays 5000.
Enter Q	Invalid transaction type.

Sample output screenshot (illustrative)

```
Starting balance: 5000
Enter transaction type: D
Enter amount: 1000
Deposit successful
Enter transaction type: W
Enter amount: 500
Withdrawal successful
Enter transaction type: X

Transaction Summary
Deposit : 1000.0
Withdrawal : 500.0
Final balance: 5500.0
```

Observation / limitation

Educational simulation only; no real banking operations.

3. Student Performance Trend Analysis

Objective

Compare marks across exams and identify weakest subject in Exam 3.

Source code

```
subjects = ["Python", "Mathematics", "English"]
exam1 = [65, 55, 70]
exam2 = [72, 60, 68]
exam3 = [80, 58, 75]
print("Student Performance Analysis")
for i in range(len(subjects)):
    change = exam3[i] - exam1[i]
    print("\nSubject:", subjects[i])
    print("Exam 1:", exam1[i])
    print("Exam 2:", exam2[i])
    print("Exam 3:", exam3[i])
    print("Overall change:", change)
    if change > 0:
        print("Performance improved")
    elif change < 0:
        print("Performance declined")
    else:
        print("No overall change")
    if exam3[i] < 40:
        print("Needs improvement")
weakest = exam3.index(min(exam3))
print("\nWeakest subject:", subjects[weakest])
print("Average in Exam 3:", sum(exam3) / len(exam3))
```

Test cases

Input / condition	Expected result
Provided sample data	Changes: Python +15, Mathematics +3, English +5.
Exam 3 mark below 40	Prints 'Needs improvement'.
Find weakest subject	Mathematics; Exam 3 average 71.0.

Sample output screenshot (illustrative)

```
Student Performance Analysis
Subject: Python
Overall change: 15
Performance improved
Subject: Mathematics
Overall change: 3
Performance improved
Subject: English
Overall change: 5
Performance improved
Weakest subject: Mathematics
Average in Exam 3: 71.0
```

Observation / limitation

Overall change compares Exam 3 with Exam 1.

4. Repeated Transaction Detection

Objective

Detect repeated transaction IDs and amounts above 10000.

Source code

```
ids = ["T101", "T102", "T103", "T101", "T104"]
amounts = [2000, 15000, 5000, 2000, 12000]
seen = []
print("Transaction Audit")
for i in range(len(ids)):
    print("\nID:", ids[i], "Amount:", amounts[i])
    if ids[i] in seen:
        print("Warning: Duplicate transaction ID")
    else:
        seen.append(ids[i])
    if amounts[i] > 10000:
        print("Warning: Amount exceeds 10000")
print("\nAudit completed")
```

Test cases

Input / condition	Expected result
T101 appears twice	Second T101 is flagged as duplicate.
Amount 15000	Amount warning appears.
Amount exactly 10000	No warning; condition is greater than 10000.

Sample output screenshot (illustrative)

```
Transaction Audit
ID: T101 Amount: 2000
ID: T102 Amount: 15000
Warning: Amount exceeds 10000
ID: T103 Amount: 5000
ID: T101 Amount: 2000
Warning: Duplicate transaction ID
ID: T104 Amount: 12000
Warning: Amount exceeds 10000
Audit completed
```

Observation / limitation

Sample records are hard-coded.

5. Password Policy Validation Engine

Objective

Check length and uppercase, lowercase, digit, and special-character requirements.

Source code

```
password = input("Enter a sample password: ")
has_upper = has_lower = has_digit = has_special = False
for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    else:
        has_special = True
valid = True
if len(password) < 8:
    print("Failed: Minimum 8 characters required")
    valid = False
if not has_upper:
    print("Failed: Add an uppercase letter")
    valid = False
if not has_lower:
    print("Failed: Add a lowercase letter")
    valid = False
if not has_digit:
    print("Failed: Add a digit")
    valid = False
if not has_special:
    print("Failed: Add a special character")
    valid = False
if valid:
    print("Password meets the sample policy")
else:
    print("Password does not meet the policy")
```

Test cases

Input / condition	Expected result
Abcd123!	Passes all checks.
abcdefg	Fails length, uppercase, digit, and special-character checks.
ABCDEFG1!	Fails lowercase check.

Sample output screenshot (illustrative)

```
Enter a sample password: Abcd123!
Password meets the sample policy
```

Observation / limitation

Use only sample passwords in demonstrations, never real account passwords.

6. String-Based Examination Answer Evaluation

Objective

Count correct, wrong, and blank answers and calculate a sample score.

Source code

```
key = ["A", "B", "C", "D", "A"]
answers = ["A", "C", "", "D", "B"]
correct = wrong = unanswered = 0
for i in range(len(key)):
    if answers[i] == "":
        unanswered += 1
    elif answers[i].upper() == key[i]:
        correct += 1
    else:
        wrong += 1
# Sample marking: +1 correct, -0.25 wrong
score = correct - (wrong * 0.25)
print("Correct answers:", correct)
print("Wrong answers:", wrong)
print("Unanswered:", unanswered)
print("Final score:", score)
```

Test cases

Input / condition	Expected result
Given sample list	2 correct, 2 wrong, 1 unanswered; score 1.5.
All correct	Score equals number of questions.
All blank	0 correct, 0 wrong, 5 unanswered; score 0.

Sample output screenshot (illustrative)

```
Correct answers: 2
Wrong answers: 2
Unanswered: 1
Final score: 1.5
```

Observation / limitation

Marking rule is illustrative; change it to match exam rules.

7. Text Compression Using Character Runs

Objective

Encode consecutive repeated characters using run-length counts.

Source code

```
text = input("Enter a string: ")
if text == "":
    print("Please enter a non-empty string")
else:
    encoded = ""
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1
    encoded += text[-1] + str(count)
    print("Original string:", text)
    print("Encoded string:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))
    if len(encoded) < len(text):
        print("Compression is beneficial")
    else:
        print("Compression is not beneficial")
```

Test cases

Input / condition	Expected result
aaaaab	Encoded a5b1; length 4 vs 6; beneficial.
aaabbc	Encoded a3b2c1; length 6 vs 6; not beneficial.
Empty string	Prompts for non-empty input.

Sample output screenshot (illustrative)

```
Enter a string: aaaaab
Original string: aaaaab
Encoded string: a5b1
Original length: 6
Encoded length: 4
Compression is beneficial
```

Observation / limitation

Simple educational encoding; special cases need a fuller format.

8. Student Marks Matrix Analysis

Objective

Compute student totals and averages, performance labels, and subject averages.

Source code

```
subjects = ["Python", "Maths", "English"]
students = ["Ravi", "Priya", "Arun"]
marks = [[85, 78, 90], [92, 88, 84], [70, 65, 75]]
print("Student Marks Analysis")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    print("\nStudent:", students[i])
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))
    if average >= 85:
        print("Performance: Excellent")
    elif average >= 70:
        print("Performance: Good")
    else:
        print("Performance: Needs improvement")
print("\nSubject Averages")
for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
        total += marks[i][j]
    average = total / len(students)
    print(subjects[j], ":", round(average, 2))
```

Test cases

Input / condition	Expected result
Ravi 85,78,90	Total 253; average 84.33; Good.
Priya 92,88,84	Total 264; average 88.0; Excellent.
Subject averages	Python 82.33; Maths 77.0; English 83.0.

Sample output screenshot (illustrative)

```
Student: Ravi
Marks: [85, 78, 90]
Total: 253
Average: 84.33
Performance: Good
Student: Priya
Marks: [92, 88, 84]
Total: 264
Average: 88.0
Performance: Excellent
Subject Averages
Python : 82.33
Maths : 77.0
English : 83.0
```

Observation / limitation

Assumes all marks are valid and rows align with subjects.

9. Food Waste Monitoring System

Objective

Compute leftover meals and daily waste percentages and report totals.

Source code

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
prepared = [100, 120, 110, 90, 130]
served = [90, 105, 100, 85, 110]
limit = 10
print("Food Waste Report")
for i in range(len(days)):
    leftover = prepared[i] - served[i]
    if prepared[i] <= 0 or leftover < 0:
        print(days[i], ": Invalid data")
        continue
    waste_percentage = leftover / prepared[i] * 100
    print("\nDay:", days[i])
    print("Prepared:", prepared[i])
    print("Served:", served[i])
    print("Leftover:", leftover)
    print("Waste percentage:", round(waste_percentage, 2), "%")
    if waste_percentage > limit:
        print("Warning: Waste exceeds limit")
    else:
        print("Waste is within limit")
total_prepared = sum(prepared)
total_served = sum(served)
total_leftover = total_prepared - total_served
print("\nTotal meals prepared:", total_prepared)
print("Total meals served:", total_served)
print("Total leftover meals:", total_leftover)
```

Test cases

Input / condition	Expected result
Monday 100 prepared, 90 served	10 leftover; 10%; within limit because condition is >10.
Tuesday 120 prepared, 105 served	15 leftover; 12.5%; warning.
Weekly totals	550 prepared, 490 served, 60 leftover.

Sample output screenshot (illustrative)

```
Food Waste Report
Day: Monday
Prepared: 100
Served: 90
Leftover: 10
Waste percentage: 10.0 %
Waste is within limit
Day: Tuesday
Prepared: 120
Served: 105
Leftover: 15
Waste percentage: 12.5 %
Warning: Waste exceeds limit
Total meals prepared: 550
Total meals served: 490
Total leftover meals: 60
```

Observation / limitation

Illustrative fixed weekday data; invalid rows are not excluded from final totals.

10. Energy Conservation Before-and-After Analysis

Objective

Compare appliance use before/after, compute savings, and identify largest unit saving.

Source code

```
appliances = ["Fan", "Light", "AC", "Computer"]
before = [30, 20, 100, 40]
after = [24, 12, 75, 32]
savings = []
print("Energy Conservation Report")
for i in range(len(appliances)):
    saved = before[i] - after[i]
    savings.append(saved)
    if before[i] > 0:
        percentage = saved / before[i] * 100
    else:
        percentage = 0
    print("\nAppliance:", appliances[i])
    print("Before:", before[i], "units")
    print("After:", after[i], "units")
    print("Units saved:", saved)
    print("Savings percentage:", round(percentage, 2), "%")
    if saved > 0:
        print("Energy consumption reduced")
    elif saved < 0:
        print("Energy consumption increased")
    else:
        print("No change in consumption")
total_before = sum(before)
total_after = sum(after)
total_saved = total_before - total_after
print("\nTotal before:", total_before, "units")
print("Total after:", total_after, "units")
print("Total energy saved:", total_saved, "units")
best = savings.index(max(savings))
print("Appliance with highest unit savings:", appliances[best])
```

Test cases

Input / condition	Expected result
Fan 30 before, 24 after	6 units saved; 20%.
AC 100 before, 75 after	25 units saved; 25%; highest unit savings.
Totals	190 before, 143 after, 47 saved.

Sample output screenshot (illustrative)

```
Energy Conservation Report
Appliance: Fan
Before: 30 units
After: 24 units
Units saved: 6
Savings percentage: 20.0 %
Appliance: AC
Before: 100 units
After: 75 units
Units saved: 25
Savings percentage: 25.0 %
Total before: 190 units
Total after: 143 units
Total energy saved: 47 units
Appliance with highest unit savings: AC
```

Observation / limitation

Units are illustrative sample values, not real meter readings.

Overall conclusion

These ten beginner-level Python case studies demonstrate conditionals, loops, lists, string processing, arithmetic, nested-list traversal, counters, validation, and summary reporting. The test cases cover representative expected behavior. Before submission, execute the code locally, verify the outputs, and capture genuine screenshots if required by the assignment rubric.

Final submission checklist

- Run every program with Python 3.
- Compare actual results with the test cases.
- Check boundary and invalid inputs.
- Capture screenshots from your own IDE or terminal if required.
- Verify project titles against the instructor's case-study sheet.